

What we're going to cover:

- Reading / writing files
- CSV
- JSON

Opening a File

`open()` تابع

برای باز کردن یک فایل استفاده میکنیم

```
In [3]: f = open("files/open.txt")
print(f)
```

```
Out[3]: <_io.TextIOWrapper name='files/open.txt' mode='r' encoding='cp1252'>
```

`read()` تابع

برای خواندن فایل استفاده می‌کنیم

```
In [4]: print(f.read())
```

```
Hi. Wellcome to python programming language. Some other dummy text
```

Closing a File

`file.close()` متد

فایل را می‌بندد و منابع سیستم را آزاد می‌کند

```
In [6]: file = open("files/open.txt", 'r')
content = file.read()
print(content)
file.close()
```

```
Hi. Wellcome to python programming language. Some other dummy text
```

Reading Files:

: پایتون روش‌های متعددی برای خواندن داده‌ها از یک فایل ارائه می‌دهد:

- `read()` :
 - کل فایل یا تعداد مشخصی از کاراکترها را می‌خواند
- `readline()` :

- یک خط از فایل را می‌خواند
- `readlines()` :
 - تمام خطوط فایل را می‌خواند و آن‌ها را در قالب یک لیست برمی‌گرداند

```
In [13]: # Reading the entire file
file = open('files/example.txt', 'r')
content = file.read()
print(content)
file.close()

# Reading the first 10 characters
file = open('files/example.txt', 'r')
content = file.read(10)
print(content)
file.close()

# Reading a single line
file = open('files/example.txt', 'r')
line = file.readline()
print(line)
file.close()

# Reading all lines
file = open('files/example.txt', 'r')
lines = file.readlines()
print(lines)
file.close()
```

```
Hello, World!This line is appended.
Hello, Wor
Hello, World!This line is appended.
['Hello, World!This line is appended.']
```

Writing to Files:

- `write()` :
 - یک رشته را در فایل می‌نویسد
- `writelines()` :
 - فهرستی از رشته‌ها را در فایل می‌نویسد

```
In [14]: # Writing a string to a file
file = open('files/example_write.txt', 'w')
file.write('Hello, world!')
file.close()

# Writing a List of strings to a file
lines = ['Hello, world!', 'Python is great!', 'File handling is easy.']
file = open('files/example_write.txt', 'w')
```

```
file.writelines(lines)
file.close()
```

Using **with** Statement:

به جای باز و بسته کردن دستی فایل، می‌توانید از دستور **with** استفاده کنید

این کار خطر خرابی فایل و نشت منابع را کاهش می‌دهد

حالت های رایج پارامتر **mode**

- **r** file is opened in **read** mode.
- **w** file is opened in **write** mode.
- **a** file is opened in **append** mode.

```
In [7]: # Reading the entire file content
with open("files/open.txt", "r") as file:
    content = file.read()
    print(content) # File closes automatically
```

Hi. Wellcome to python programming language. Some other dummy text

```
In [8]: # Reading line by line
with open("files/open.txt", "r") as file:
    for line in file:
        print(line)
```

Hi. Wellcome to python programming language. Some other dummy text

```
In [9]: # Writing a string to a file
with open("files/example.txt", "w") as file:
    file.write("Hello, World!")
```

```
In [12]: with open("files/geek.txt", "w") as file:
        file.write("Hello, Python!\n")
        file.write("File handling is easy with Python.")

        print("File written successfully")
```

File written successfully

```
In [10]: # Writing multiple lines to a file
lines = ["Line 1", "Line 2", "Line 3"]
with open("files/example2.txt", "w") as file:
    for line in lines:
        file.write(line + "\n")
```

```
In [11]: # Appending to an existing file
with open("files/example.txt", "a") as file:
    file.write("This line is appended.")
```

The CSV file

Comma Separated Values

فرمت CSV روشی رایج و ساده برای ذخیره‌سازی داده‌های جدولی است
برای نمایش یک فایل CSV آن فایل باید دارای پسوند csv باشد

```
In [23]: ### Import the CSV module  
import csv
```

Write into CSV files with `csv.writer()`

```
In [24]: # newline = creates an empty line between each line we write  
  
with open('files/innovators.csv', 'w', newline='') as file:  
    writer = csv.writer(file)  
    writer.writerow(["SN", "Name", "Contribution"])  
    writer.writerow([1, "Linus Torvalds", "Linux Kernel"])  
    writer.writerow([2, "Tim Berners-Lee", "World Wide Web"])  
    writer.writerow([3, "Guido van Rossum", "Python Programming"])
```

Writing Multiple Rows with `writerows()`

```
In [25]: row_list = [ ["SN", "Name", "Contribution"],  
                     [1, "Linus Torvalds", "Linux Kernel"],  
                     [2, "Tim Berners-Lee", "World Wide Web"],  
                     [3, "Guido van Rossum", "Python Programming"] ]  
  
with open('files/protagonist.csv', 'w') as file:  
    writer = csv.writer(file)  
    writer.writerows(row_list)
```

CSV Files with Custom Delimiters

به‌طور پیش‌فرض، در فایل‌های CSV از ویرگول به عنوان جداکننده استفاده می‌شود
با این حال، برخی از CSV ممکن است از جداکننده‌هایی غیر از ویرگول استفاده کنند
فایل‌های

Few popular ones are `|` and `\t`.

```
In [26]: data_list = [ ["SN", "Name", "Contribution"],  
                      [1, "Linus Torvalds", "Linux Kernel"],  
                      [2, "Tim Berners-Lee", "World Wide Web"],  
                      [3, "Guido van Rossum", "Python Programming"] ]  
  
with open('files/innovators_pipe_delimiter.csv', 'w', newline='') as file:  
    writer = csv.writer(file, delimiter='|')  
    writer.writerows(data_list)
```

Read CSV files with `csv.reader()`

```
In [28]: with open('files/innovators.csv', 'r') as file:
        reader = csv.reader(file)
        for row in reader:
            print(row)
```

```
['SN', 'Name', 'Contribution']
['1', 'Linus Torvalds', 'Linux Kernel']
['2', 'Tim Berners-Lee', 'World Wide Web']
['3', 'Guido van Rossum', 'Python Programming']
```

Read CSV files with `csv.DictReader()`

```
In [30]: with open("files/people.csv", 'r') as file:
        csv_file = csv.DictReader(file)
        for row in csv_file:
            print(dict(row))
```

```
{'Name': 'Jack', 'Age': '23', 'Profession': 'Doctor'}
{'Name': 'Miller', 'Age': '22', 'Profession': 'Engineer'}
```

JSON

JavaScript Object Notation

یک فرمت داده محبوب برای نمایش داده‌های ساختاریافته است

تبادل داده‌ها (ارسال و دریافت) میان سرور و برنامه تحت وب با JSON امری رایج است
فرمت

```
In [ ]: # In Python, JSON exists as a string. For example:
        p = '{"name": "Bob", "languages": ["Python", "Java"]}'
```

```
In [31]: import json
```

Python JSON to dict

`json.loads()` متد

می‌توانید یک رشته JSON تجزیه کنید. این متد یک دیکشنری برمی‌گرداند

```
In [34]: person = '{"name": "Bob", "languages": ["English", "French"]}'
        person_dict = json.loads(person)
        print(person_dict)
        print('--' * 10)
        print(person_dict['languages'])
```

```
{'name': 'Bob', 'languages': ['English', 'French']}
```

```
-----  
['English', 'French']
```

Python read JSON file

`json.load()` متد

برای خواندن فایلی که حاوی یک شیء JSON است، استفاده کنید

```
In [35]: # Suppose, you have a file named person.json which contains a JSON object.
```

```
with open('files/person.json', 'r') as f:  
    data = json.load(f)  
  
print(data)
```

```
{'name': 'Bob', 'languages': ['English', 'French']}
```

Python Convert to JSON string

`json.dumps()` متد

یک دیکشنری را به رشته JSON تبدیل کنید

```
In [36]: person_dict = {  
    'name': 'Bob',  
    'age': 12,  
    'children': None  
}  
person_json = json.dumps(person_dict)  
print(person_json)
```

```
{"name": "Bob", "age": 12, "children": null}
```

Writing JSON to a file

`json.dump()` متد

برای نوشتن JSON در یک فایل در پایتون، می‌توانیم استفاده کنیم

```
In [2]: import json  
person_dict = {"name": "Bob",  
               "languages": ["English", "French"],  
               "married": True,  
               "age": 32  
}  
  
with open('files/person_json.txt', 'w') as json_file:  
    json.dump(person_dict, json_file)
```